SYNARTESIS

An undo layer for AI agents

A user guide, from install to undo.

Version **0.3.2** · September 2026

WWW.LINKEDIN.COM/IN/ARHAANKHAN143

What this is for

An agent with write access to a real system runs twenty steps, misreads step seven, and applies the rest to the wrong records. Your options today are to reverse it by hand from the transcript, restore a backup and lose every legitimate change made in the same window, or accept the damage.

Synartesis sits between your AI client and the servers it calls. It records every tool call together with the state that call replaced, and it can put that state back. What cannot be put back, it refuses to let an agent do unsupervised.

It is not a sandbox. The container your agent runs in is disposable; the CRM row it updated over the network is not. It is not a tracing tool. A trace tells you `update_customer` ran forty times, not what the values were before.

THE FIVE MINUTES AHEAD OF YOU

1. Install it.
2. Point it at one MCP server. It writes a policy for you.
3. Read the policy.
4. Point your AI client at Synartesis instead of at the server.
5. Leave the watch screen open while your agent works.

Steps 6 onward are what you do when something goes wrong, which is the reason you installed it.

Step 1. Install

WHAT YOU NEED FIRST

TOOL	VERSION	CHECK WITH
Node	22 or newer	`node --version`
npm	ships with Node	`npm --version`

If `node --version` prints something lower than v22, update Node before going on. Nothing below 22 will work.

INSTALL IT

```
npm install -g synartesis
```

Confirm it landed:

```
synartesis --version
```

```
0.3.2
```

If your shell answers `command not found`, npm's global bin directory is not on your `PATH`. Find it with `npm prefix -g`, then add `<that path>/bin` to your `PATH`.

WHAT JUST GOT INSTALLED

Two commands.

- `synartesis` is the one you type. Everything in this guide uses it.
- `synartesis-proxy` is the one your AI client starts. You will name it once, in a config file, and then never think about it again.

Step 2. Point it at a server

`init` starts an MCP server, asks it what tools it has, and writes a policy saying what may be undone and how.

The example below covers a folder of files, because it is the server most people have to hand. Substitute your own server's start command after the `--`.

```
synartesis init files -- npx -y @modelcontextprotocol/server-filesystem ~/notes
```

The first run takes a moment while `npx` fetches the server.

```
  W R O T E  /Users/you/.synartesis/synartesis.yaml

  Recognised 14 tools, so the policy that ships for filesystem was used.
  Read it before you trust it, then point your MCP client at:

  synartesis proxy --manifest /Users/you/.synartesis/synartesis.yaml
```

READING THAT MESSAGE

`files` is a name you chose. It is how this server is referred to everywhere else: in the policy, in the journal, in `files.write_file`. Pick something short.

`Recognised 14 tools` means Synartesis knew this server and used a finished policy that ships with it, after checking every rule against the tools the server actually advertises. For a server it does not know, you get a draft instead where every tool is marked `irreversible` with a `TODO`. See "Writing a policy yourself" near the end.

The path it printed is the policy. You are about to read it.

WHERE THINGS LIVE

Unless a policy already sits in the directory you are standing in or above it, Synartesis keeps everything in one place:

```
~/.synartesis/
    synartesis.yaml     the policy: what each tool does and how to undo it
    journal.db          the record of everything an agent has done
```

Set `SYNARTESIS_HOME` to put that somewhere else. A policy that belongs to one project can instead sit inside it, and is found from any directory within that project the way a version control tool finds its root.

`init` adds to an existing policy rather than replacing it, so you can run it again for a second server.

Step 3. Read what it wrote

```
synartesis check
```

```
  P O L I C Y  /Users/you/.synartesis/synartesis.yaml

  servers  files
  policies 10 readonly, 2 reversible, 2 irreversible
  guarded  2

  Anything not mentioned here is treated as irreversible and guarded.
```

`check` confirms the policy names tools that actually exist. Run it after any edit. It is faster than finding out from a failed call.

THE FOUR CLASSES

Every tool gets exactly one. This is the whole idea, so it is worth two minutes.

CLASS	MEANING	EXAMPLE	WHAT SYNARTESIS DOES
`readonly`	Changes nothing	`read_text_file`	Records it, forwards it
`reversible`	Prior state can be restored exactly	`write_file`	Reads the old state first, writes it back on undo
`compensable`	Cannot be reversed, but can be offset	`create_entities`	Calls something else that neutralises it
`irreversible`	Neither	`create_directory`	**Holds it until a person says yes**

That last line about anything not mentioned is the important one. A tool your policy does not name is treated as `irreversible` and held. This is deliberate: silently forwarding an unknown destructive call is the one failure worth avoiding most.

OPEN THE FILE

```
open ~/.synartesis/synartesis.yaml
```

You do not have to understand all of it today. You do need to know it exists and that it is yours to edit, because a policy you have not read is a promise you have not checked.

Step 4. Point your AI client at Synartesis

This is the step people get wrong, so it is spelled out per client.

The idea: wherever your client currently lists an MCP server, you replace that entry with Synartesis. Your agent sees the same tools under the same names returning the same results. Nothing about how you work changes.

CLAUDE DESKTOP

Find the file. On macOS:

```
~/Library/Application Support/Claude/claude_desktop_config.json
```

On Windows:

```
%APPDATA%\Claude\claude_desktop_config.json
```

Open it. You will find a `mcpServers` block. Paste this inside it, replacing `/Users/you` with your own home directory:

```
{
  "mcpServers": {
    "synartesis": {
      "command": "synartesis-proxy",
      "args": [
        "--manifest",
        "/Users/you/.synartesis/synartesis.yaml"
      ]
    }
  }
}
```

If an entry for the server you just wrapped is already in that file, **delete it**. Leaving it means your agent can still reach the server directly, going around Synartesis entirely, and nothing will be recorded.

Use the full absolute path to the policy. A relative path is read against whatever directory the client happened to start in, which is not somewhere you can predict.

Quit Claude Desktop completely and reopen it. It reads this file only at start.

CLAUDE CODE

One command, from your project directory:

```
claude mcp add synartesis synartesis-proxy --manifest /Users/you/.synartesis/synartesis.yaml
```

Or write `.mcp.json` in the project root by hand, using the same JSON shape as the Claude Desktop example above.

ANY OTHER MCP CLIENT

Every client has somewhere it lists MCP servers, and almost all of them use the same `mcpServers` shape shown above. Two things matter wherever you put it:

- the command is `synartesis-proxy`
- `--manifest` takes the absolute path that `init` printed

CHECK THAT IT WORKED

Ask your agent to do something harmless, such as reading a file. Then:

```
synartesis list
```

```
  R U N S  most recent first

  run                                  started                  status      actions  agent
  adbef700-7f5d-4c88-a69c-149ef1d62730  2026-09-01T11:16:09.000Z  complete          2  my-agent
```

A run appears the first time an agent calls a tool through the proxy. If the list is empty, your client is still talking to the server directly. Go back and check you removed the old entry.

Step 5. Leave the watch screen running

```
synartesis watch
```

This is the one to keep open in a second terminal while an agent works. Anything held for approval appears here, and `a` and `d` answer it without a second terminal or an id to copy.

KEY	DOES
`j` / `k`	Move between runs
`Enter`	Open a run
`p`	Preview an undo without doing it
`u`	Undo the selected run
`g`	Show what is held
`a` / `d`	Approve or deny
`q`	Quit

Everything acts on whatever the cursor is on, so no id is ever carried from one command to the next by hand.

Typing `synartesis` with no arguments opens the same screen.

Step 6. When a call is held

Your agent tries something irreversible. The call does not go through. It is refused straight away with the command that approves it, rather than left hanging, because every window a person needs is longer than a client will wait.

See what is waiting:

```
synartesis gates
```

```
  A W A I T I N G   A P P R O V A L

  08f8d6fd-8123-4dae-9b35-abb0ce66553a  2026-09-01T11:16:09.003Z
  files.create_directory  {"path":"/Users/you/notes/junk"}
  irreversible  this action cannot be undone

  synartesis approve 08f8d6fd --by <name>
  synartesis approve --all --by <name>
```

It prints the exact command back to you, with the id already shortened. Ids may be shortened to any unambiguous prefix.

ALLOWING IT

```
synartesis approve 08f8d6fd --by arhaan
```

Approving does not perform the call. It permits it. Your agent makes the same call again and this time it goes through.

REFUSING IT

```
synartesis deny --all --by arhaan --reason "not needed"
```

```
  denied files.create_directory 08f8d6fd-8123-4dae-9b35-abb0ce66553a
```

The reason stays on the record.

`--by` is who is deciding. It defaults to the logged-in user; give it explicitly when more than one person can answer.

Step 7. Undo

This is what you installed it for.

LOOK BEFORE YOU LEAP

```
synartesis undo --dry-run
```

```
  D R Y   R U N  adbef700-7f5d-4c88-a69c-149ef1d62730

    2  skip             files.create_directory  never applied (denied)
    1  revert           files.write_file  state matches; applying inverse
       would call files.write_file {"path":".../notes/roadmap.md", ...}

  R E S U L T  rolled_back
```

`--dry-run` re-reads the current state and prints the plan without changing anything. `state matches` means the resource is still as that step left it, so the undo is safe.

DO IT

```
synartesis undo
```

```
  U N D O  adbef700-7f5d-4c88-a69c-149ef1d62730

    2  skip             files.create_directory  never applied (denied)
    1  revert           files.write_file  state matches; applying inverse
       called files.write_file {"path":".../notes/roadmap.md", ...}

  R E S U L T  rolled_back
```

With no run id, `undo` takes the most recent run. Give an id, or a prefix of one, to reach an older one.

Undo walks a run **backwards, newest call first**. That order is what makes a partial undo safe: whatever it has already put back stays put back.

WHEN IT REFUSES

Before reversing a step, Synartesis re-reads the resource and compares it against the state that step left behind. If somebody has been there since, it stops:

```
  halted at sequence 1  drift detected
  drift at sequence 1: the resource is not in the state this run left it in.
    at line 1:
    - AGENT VERSION
    + A HUMAN FIXED THIS BY HAND
    1 removed, 1 added.

  R E S U L T  partial
```

This is the feature, not a failure. Writing the old value back would have destroyed a colleague's work. It shows you the lines that differ and stops.

`partial` means some steps were reversed and one was not. Resolve the conflict, then carry on from where it stopped:

```
synartesis undo --replan
```

USEFUL FLAGS

FLAG	DOES
`--dry-run`	Print the plan, change nothing
`--to <seq>`	Lowest sequence to undo; earlier steps are left alone
`--replan`	Rebuild each undo from the current policy
`--json`	Machine-readable output, for `list`, `show` and `gates`

SEEING ONE RUN IN DETAIL

```
synartesis show
```

```
  T I M E L I N E

    1  <- reversible   applied        files.write_file
       {"path":"/Users/you/notes/roadmap.md", ...}
       undo: {"server":"files","tool":"write_file","args":{...}}
    2  !  irreversible gated          files.create_directory
       {"path":"/Users/you/notes/junk"}
       note: this action cannot be undone

  2 actions: 1 applied, 1 gated | 1 with a recorded undo
```

Every step, with the exact call that would reverse it.

Step 8. Housekeeping

THE JOURNAL GROWS

Putting a file back means keeping what was in it. Synartesis stores the resource as it was, as it became, and the call that did it, so the journal grows at roughly **four times the bytes your agent writes**, and never shrinks by itself. Thirty edits of one 200 kB file came to 24 MB.

TRIMMING IT

```
synartesis prune --dry-run
```

```
  /Users/you/.synartesis/journal.db

  my-agent                2026-09-01 11:16:21  rolled_back 2 actions

  1 runs and 2 actions would go. Nothing was changed.
```

Then for real:

```
synartesis prune
```

Deletes whole runs finished more than 30 days ago and reclaims the space. `--older-than <days>` changes the cutoff.

Nothing still active or still waiting on a person is ever pruned. Age is not a reason to discard a run whose outcome nobody knows, or one holding a decision somebody has not made.

`prune` deletes irreversibly and does not ask twice. Run `--dry-run` first.

THE JOURNAL HOLDS YOUR FILE CONTENTS

To put a file back, Synartesis must first store what was in it. A key that was sitting in a file your agent touched is in there in plain text. That is not a leak to be closed; it is the thing that makes undo work.

So it is treated as private data. The journal is created `0600`, the directory Synartesis makes for it `0700`, and `0600` is re-applied every time a journal is opened. A directory that already existed is left alone, because a journal can sit beside a policy inside a project and silently making your project directory `0700` would be the worse surprise.

If your `~/.synartesis` predates version 0.3.0, run this once:

```
chmod 700 ~/.synartesis
```

Nothing is encrypted. Full-disk encryption answers a stolen laptop; file permissions answer another account on a machine you share.

Writing a policy yourself

For servers Synartesis does not recognise, `init` writes a draft where every tool that is not a self-declared read starts as `irreversible` with a `TODO`. Working through those TODOs is the job.

A rule needs three things:

```
servers:
  crm:
    command: node
    args: ["server.js"]

tools:
  crm.update_customer:
    class: reversible
    snapshot:
      tool: get_customer
      args: { id: "${args.id}" }
    inverse:
      tool: update_customer
      args:
        id: "${args.id}"
        plan: "${snapshot.plan}"
        notes: "${snapshot.notes}"
```

- **`snapshot`** is the read that captures the old state, before the write goes out. If this read fails, the write does not happen. A reversible action without a snapshot is only silently irreversible.
- **`inverse`** is the call that puts it back, built from what the snapshot returned.

Run `synartesis check` after every edit.

Four finished policies ship with Synartesis, for filesystem, memory, git and github. `init` uses them automatically when it recognises the server. They are worth reading as worked examples.

Troubleshooting

`command not found: synartesis` npm's global bin is not on your `PATH`. Run `npm prefix -g` and add `/bin` to it.

`synartesis list` is empty after the agent worked Your client is still talking to the server directly. Check you removed the original entry from the config, and that you restarted the client.

A run is stuck at `active` A proxy was killed mid-run. Nothing guesses at this, since several proxies can share one journal. Close it by hand:

```
synartesis close
```

`fs is already declared in the manifest` You ran `init` twice with the same name. `init` adds to a policy rather than replacing it. Pick another name, or edit the policy and remove the old entry.

Undo halted with `drift detected` Working as intended. Somebody changed the resource after your agent did. Read the diff it printed, resolve it, then `synartesis undo --replan`.

A tool you expected to work is being held It is not in your policy, so it is treated as irreversible. Add a rule for it, or approve it each time.

The journal is enormous `synartesis prune --dry-run`, then `synartesis prune`.

Command reference

COMMAND	DOES
`synartesis`	The screen, driven with arrow keys
`synartesis init <name> -- <command>`	Ask a server what it can do, draft a policy
`synartesis check`	Confirm the policy names tools that exist
`synartesis list`	Every run, most recent first
`synartesis show [run]`	One run, step by step, with each undo
`synartesis gates`	What is waiting on a person
`synartesis watch`	The live screen, for a second terminal
`synartesis approve [id|--all]`	Let a held call through
`synartesis deny [id|--all]`	Refuse it, with a reason
`synartesis undo [run]`	Walk a run backwards, newest first
`synartesis close [run]`	End a run left open by a killed proxy
`synartesis prune`	Delete old runs and reclaim the space
`synartesis proxy --manifest <path>`	What your client runs, not you

Global flags: `--manifest`, `--journal`, `--json`, `--dry-run`, `--version`, `--help`.

Neither path usually needs giving.

What it will not do

It cannot un-send what has been seen. An email that was read, a message somebody screenshotted, a file deleted where nothing keeps backups. That is why the gate exists, rather than a promise it could not keep.

It only sees what goes through MCP. An agent making its own HTTP calls, running a shell command, or driving a browser outside the protocol is invisible to it. Synartesis intercepts a protocol, not an intention.

It stops rather than guessing. Undo halts at the first step it cannot reverse honestly, leaving a clean partial state and telling you where it stopped.

It cannot snapshot something very large. A resource over roughly three megabytes cannot be read back through a stdio connection. Synartesis says so and refuses the write rather than applying a change it could not capture. Nothing is lost; the call does not go through.

It can only undo what the system underneath allows. GitHub has no delete for issues, so creating one is guarded, not reversible. That is the shape of the world, and the job is to be honest about it.

---

synartesis.online · github.com/ArhaanDev24/Synartesis · npm i -g synartesis